

# CSEN 296B-2: Week 2.1 Definition of Done

## AstraNotes Project

Student: Atishay Jain | Date: April 6, 2026 | Technical Path: Python

### 1. Definition of Done for AstraNotes

A task, feature, or artifact in the AstraNotes project is considered Done only when every applicable criterion in the table below is satisfied. These criteria apply equally to work produced individually and to work produced with AI assistance. No item moves forward into integration, a new sprint, or a submission unless the full checklist has been applied.

| Criterion                       | What Must Be True Before the Work Moves Forward                                                                                                                                                                                                                                                                                                                   |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Requirement Traceability     | The task maps explicitly to a requirement ID (FR01 through FR08, NFR01 through NFR03, or SPR01 through SPR04) from the Week 1.2 requirements submission, or to a documented backlog item. Work that cannot be traced to an existing requirement is either rejected or triggers a formal requirement update before proceeding.                                     |
| 2. Explainability               | I can describe the logic, design decision, or implementation approach in my own words without referring back to AI output. If I cannot explain it without the AI response in front of me, it is not done.                                                                                                                                                         |
| 3. Architecture Alignment       | The output is consistent with the layered architecture defined in the Week 1.2 architecture submission: Note Entity, Repository Interface, JSON File Adapter, Privacy Service, Validation Layer, Version History Service, and Note Manager. Any deliberate deviation from this architecture is logged as a new Architecture Decision before the work is accepted. |
| 4. Quality and Realism Check    | The output has been reviewed for clarity, correctness, and feasibility within the constraints of a solo-developer, one-quarter project. AI-generated suggestions that introduce unnecessary complexity or external dependencies beyond what the project requires are rejected at this gate.                                                                       |
| 5. Privacy and Security Gate    | For any work that touches note storage, retrieval, or display: encryption at rest (SPR01) has been explicitly considered. Private note bodies are never written in plaintext. The Privacy Service boundary has not been bypassed or merged with storage logic.                                                                                                    |
| 6. Error Handling Verified      | Any component that reads from or writes to disk includes explicit handling for NoteNotFoundError, PersistenceError, and ValidationError, consistent with the error strategy in the Week 1.2 architecture submission. Clear, non-technical user-facing messages are required per SPR02.                                                                            |
| 7. Testing or Verification Plan | Either an automated test exists for the component, or a documented manual verification procedure is recorded and linked to the task, per SPR03. Work that has no verification path at all is not considered done.                                                                                                                                                 |

|                                    |                                                                                                                                                                                                                                             |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8. Prompt and Decision Log Updated | The prompt log and Architecture Decision Log have been updated to reflect any AI-assisted decisions made during this task. No consequential decision is accepted without a log entry.                                                       |
| 9. Willingness to Defend           | I would be comfortable presenting this work to the instructor and explaining every decision in it without additional preparation. If I would feel uncertain defending a piece of the work, that uncertainty is a signal it is not yet done. |

## How the Nine Criteria Apply by Work Type

The following examples show how each category of AstraNotes deliverable is evaluated against the Definition of Done. These are not exhaustive but illustrate the intent of each criterion in practice.

- **Implementation task (e.g., building the JsonFileRepository module):** All nine criteria apply. The module must trace to FR05 or FR06, be explainable without the AI response, conform to the repository interface design, handle disk errors, have at least a manual verification plan, and be logged in the prompt record.
- **Design artifact (e.g., designing VoiceNote extensibility):** Criteria 1 through 4 and 8 are primary. The design must trace to a real backlog item, be explainable, not violate the existing repository interface or privacy boundary, be realistic for this project, and be logged as an Architecture Decision.
- **Lab submission (e.g., this document):** All nine criteria apply. The submission must trace to the week's requirements, every sentence must be defensible, the architecture must be consistent with prior decisions, privacy and governance concerns must be addressed, and the prompt log must be updated.
- **Backlog or requirements change (e.g., adding a new non-functional requirement):** Criteria 1, 2, 4, and 8 are primary. The change must be justified by a real project need, not just an AI suggestion. It must not conflict with any existing requirement, and the decision must be logged before the backlog is updated.

---

## 2. Applying the Definition of Done to Prior AstraNotes Work

To confirm that this Definition of Done is grounded in the actual project and not just theoretical, I applied it retroactively to the two deliverables from Week 1.2.

### Week 1.2 Submission 1: Architecture Decision Log

- **Requirement Traceability:** Met. The architecture components (Note Entity, Repository Interface, JSON File Adapter, Privacy Service, Validation Layer, Version History Service, Note Manager) were defined to support the requirements established in the companion requirements submission.
- **Explainability:** Met. Each component has a single stated responsibility and I can describe its role without re-reading the AI output.

- **Privacy and Security Gate:** Met. The Privacy Service is an isolated component responsible for Fernet encryption and decryption. Private note bodies are never passed to the repository in plaintext.
- **Testing or Verification Plan:** Partially met. NFR03 requires that components be testable in isolation through dependency injection, but no test stubs or verification plan has been written yet. Producing a verification plan for each component is a Week 3 backlog item.
- **Willingness to Defend:** Met. The architecture verdict and reflection in the submission explain the reasoning behind each component boundary and why the refined version is stronger than the initial draft.

## Week 1.2 Submission 2: Initial Requirement Set

- **Requirement Traceability:** Met. All 15 requirements (FR01 through FR08, NFR01 through NFR03, SPR01 through SPR04) map directly to architectural components or project constraints established in Submission 1.
- **Explainability:** Met. Each requirement includes a verifiable condition (for example, under 2 seconds for NFR01, encrypted at rest for SPR01) that can be stated without AI assistance.
- **Quality and Realism Check:** Met. The requirements are scoped to a solo-developer, one-quarter Python project. No requirement introduces external services, multi-user workflows, or infrastructure beyond local file storage.
- **Prompt and Decision Log:** Partially met. The submission documents the contrast between the weak and strong AI-generated requirement drafts, but a formal prompt log entry for each requirement iteration has not been recorded separately. This will be formalized going forward using the logging standard in Section 1.3 of the Working Agreement.

---

## 3. Reflection: How This Definition of Done Will Help AstraNotes

The core purpose of this Definition of Done is to make the gap between finished-looking work and actually done work impossible to ignore. In Week 1, AI produced requirement and architecture drafts that were well-formatted and confident in tone. The first drafts were not acceptable for the project, not because they were obviously wrong, but because they were too vague to be testable, too generic to trace to AstraNotes specifically, and too broad to fit a solo developer. The DoD formalizes the standard that caught those problems and makes it apply to every piece of work going forward.

The privacy and security gate (Criterion 5) is the most critical checkpoint for AstraNotes specifically. The application stores notes that users may mark private, and the architecture depends on the assumption that the Privacy Service encrypts those bodies before they reach the repository. Any implementation task that skips this check risks introducing a plaintext storage bug that would violate SPR01. The gate makes that check mandatory and not something that can be forgotten in the interest of moving quickly.

The willingness-to-defend criterion (Criterion 9) is intentionally subjective because it catches a failure mode that checklists alone cannot catch: accepting work that looks complete but that I do not actually understand. If I would feel unprepared explaining a decision to the instructor, that

discomfort is meaningful information. The criterion makes that internal signal a formal quality gate rather than something easy to override when deadlines approach.

Applying the DoD retroactively to the Week 1.2 submissions revealed two partial gaps: the absence of a written verification plan for each architecture component, and the absence of a formalized prompt log entry for each requirements iteration. Both are now on the backlog. Identifying these gaps before the coding phase begins is exactly the kind of early correction this Definition of Done is designed to enable.